**Name: Pranav Pol Class : D20A Roll no: 44 Batch: C**

# EXPERIMENT 11

**Aim -** Implementation of Hyperledger Fabric Network and Asset Management using Chaincode.

**Theory -**

Hyperledger Fabric Network Setup

- Consortium & Membership Service Provider (MSP): Fabric is permissioned, meaning only authorized organizations can join. MSPs define identities and roles.
- Peers: Nodes that maintain the ledger and endorse transactions. Each organization typically runs peers.
- Orderer: The ordering service packages endorsed transactions into blocks and delivers them to peers.
- Ledger: Composed of a blockchain (immutable transaction log) and a state database (current values).
- Channels: Private sub-networks where a subset of organizations transact confidentially.

Chaincode (Smart Contracts)

- Chaincode defines the business logic for asset management.
- Written in Go, Java, or Node.js.
- Deployed to peers, then instantiated on a channel.
- Functions typically include createAsset, queryAsset, transferAsset, and deleteAsset.

Transaction Flow

1. Proposal Creation: A client application submits a transaction proposal (e.g., create asset).
2. Endorsement: Peers simulate the transaction using chaincode and return signed results.
3. Ordering: The orderer sequences endorsed transactions into blocks.
4. Validation & Commit: Peers validate endorsements, check for conflicts, and commit the block to the ledger.

Asset Operations

- Asset Creation:
    - Chaincode function createAsset stores asset attributes (ID, owner, value) in the ledger's state database.
- Asset Querying:
    - queryAsset retrieves asset details by key. Rich queries are possible if CouchDB is used as the state database.
- Asset Transfer:
    - transferAsset updates the owner field of an asset. The transaction is endorsed, ordered, and committed, ensuring immutability and consensus.

Key Principles

- Immutability: Once committed, transactions cannot be altered.

- Consensus: Endorsement policies ensure multiple organizations agree before a transaction is valid.
- Privacy: Channels and private data collections allow selective visibility.
- Trust: Identities are cryptographically verified via MSPs.

In essence, Hyperledger Fabric provides a modular, permissioned blockchain framework where chaincode governs asset lifecycles. The theory emphasizes identity-driven trust, modular architecture, and consensus-based validation — ensuring secure creation, querying, and transfer of assets across organizations.

**Prerequitise:** Check if WSL is Installed

Open **PowerShell (Run as Administrator)** and run:
*wsl --list --verbose*
If it shows **no distributions**, you must install Ubuntu.

**Install Ubuntu for WSL**
Run this command in **PowerShell (Admin)**:
*wsl --install -d Ubuntu*

This installs:

- Ubuntu
- WSL2 kernel

The installation may take a few minutes.
**Restart Your Computer**

After installation finishes, **restart your PC**.
**Open Ubuntu**
Open:
Start Menu → Ubuntu

It will ask you to create:
Username

Password

Now Ubuntu is ready.

**Implementation**

**Step 1 — Update the system**
**code :**
*sudo apt update*
*sudo apt upgrade -y*

**Explain:** apt update refreshes the package index — it tells your system what versions of software are available. apt upgrade installs the newer versions. Fabric's dependencies (Docker, curl, git) can fail or behave unexpectedly on outdated system libraries. Always do this first on a fresh machine.

```
pranav@DESKTOP-OSHSLJI:~$ sudo apt update
sudo apt upgrade -y
[sudo] password for pranav:
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1540 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [246 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.6 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [10.6 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [976 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [219 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.2 kB]
Get:13 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [20.8 kB]
Get:14 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [2683 kB]
Get:15 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [623 kB]
Get:16 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [208 B]
Get:17 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 c-n-f Metadata [544 B]
Get:18 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Packages [28.8 kB]
Get:19 http://security.ubuntu.com/ubuntu noble-security/multiverse Translation-en [6732 B]
Get:20 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [212 B]
Get:21 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 c-n-f Metadata [396 B]
Get:22 http://archive.ubuntu.com/ubuntu noble/universe Translation-en [5982 kB]
Get:23 http://archive.ubuntu.com/ubuntu noble/universe amd64 Components [3871 kB]
Get:24 http://archive.ubuntu.com/ubuntu noble/universe amd64 c-n-f Metadata [301 kB]
Get:25 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [269 kB]
Get:26 http://archive.ubuntu.com/ubuntu noble/multiverse Translation-en [118 kB]
Get:27 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 Components [35.0 kB]
Get:28 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 c-n-f Metadata [8328 B]
Get:29 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1837 kB]
Get:30 http://archive.ubuntu.com/ubuntu noble-updates/main Translation-en [336 kB]
Get:31 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [177 kB]
Get:32 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [16.7 kB]
```

**Step 2 — Verify prerequisites**

**code :**

*sudo apt install jq curl git nodejs npm -y*

*node -v*

*npm -v*

*git --version*

*docker --version*

*docker ps*

```
pranav@DESKTOP-OSHSLJI:~$ node -v
v18.19.1
pranav@DESKTOP-OSHSLJI:~$ npm -v
11.8.0
pranav@DESKTOP-OSHSLJI:~$ git --version
git version 2.43.0
pranav@DESKTOP-OSHSLJI:~$ docker --version
docker ps
Docker version 29.2.0, build 0b9d198
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

**Explain:** Fabric requires Docker (to run peer/orderer containers), Git (to clone the repo), and Node.js + npm (to build JavaScript chaincode). Running docker ps with no output confirms the Docker daemon is running and your user can access it. If docker ps requires sudo, you need to add your user to the docker group.

**Step 3 — Clone fabric-samples**

**code :**

*git clone https://github.com/hyperledger/fabric-samples*

*cd fabric-samples*

```
pranav@DESKTOP-OSHSLJI:~$ git clone https://github.com/hyperledger/fabric-samples
Cloning into 'fabric-samples'...
remote: Enumerating objects: 15527, done.
remote: Counting objects: 100% (98/98), done.
remote: Compressing objects: 100% (60/60), done.
remote: Total 15527 (delta 57), reused 38 (delta 38), pack-reused 15429 (from 3)
Receiving objects: 100% (15527/15527), 24.16 MiB | 6.18 MiB/s, done.
Resolving deltas: 100% (8722/8722), done.
pranav@DESKTOP-OSHSLJI:~$ cd fabric-samples
```

**Explain:** fabric-samples is the official repository containing the test-network scripts, ready-made chaincode examples (including asset-transfer-basic), and configuration files. Everything else in this guide runs from inside this directory.

**Step 4 — Download the install script and run it**

**code :**

*curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh*

*chmod +x install-fabric.sh*

*./install-fabric.sh docker samples binary*

*./install-fabric.sh docker*

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples$ curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
pranav@DESKTOP-OSHSLJI:~/fabric-samples$ chmod +x install-fabric.sh
pranav@DESKTOP-OSHSLJI:~/fabric-samples$ ./install-fabric.sh docker samples binary

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.15 does not exist, defaulting to main. fabric-samples main branch is intended to work with recent versions of fabric.

Pull Hyperledger Fabric binaries

===> Downloading version 2.5.15 platform specific fabric binaries
===> Downloading:  https://github.com/hyperledger/fabric/releases/download/v2.5.15/hyperledger-fabric-linux-amd64-2.5.15.tar.gz
===> Will unpack to: /home/pranav/fabric-samples
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
100  120M  100  120M    0     0  6272k      0  0:00:19  0:00:19 --:--:-- 6483k
==> Done.
===> Downloading version 1.5.17 platform specific fabric-ca-client binary
```

```
===> List out hyperledger images
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15    654bd4554bf9        250MB         80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17        453a0a727e82        381MB          123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15     86dded7eda92          1GB          255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15   9972c209be47        178MB         49.5MB
ghcr.io/hyperledger/fabric-peer:2.5.15      3e25adc31a75        230MB         66.2MB
hyperledger/fabric-baseos:2.5               654bd4554bf9        250MB         80.4MB
hyperledger/fabric-baseos:2.5.15            654bd4554bf9        250MB         80.4MB
hyperledger/fabric-baseos:latest            654bd4554bf9        250MB         80.4MB
hyperledger/fabric-ca:1.5                   453a0a727e82        381MB          123MB
hyperledger/fabric-ca:1.5.17                453a0a727e82        381MB          123MB
hyperledger/fabric-ca:latest                453a0a727e82        381MB          123MB
hyperledger/fabric-ccenv:2.5                86dded7eda92          1GB          255MB
hyperledger/fabric-ccenv:2.5.15             86dded7eda92          1GB          255MB
hyperledger/fabric-ccenv:latest             86dded7eda92          1GB          255MB
hyperledger/fabric-orderer:2.5              9972c209be47        178MB         49.5MB
hyperledger/fabric-orderer:2.5.15           9972c209be47        178MB         49.5MB
hyperledger/fabric-orderer:latest           9972c209be47        178MB         49.5MB
hyperledger/fabric-peer:2.5                 3e25adc31a75        230MB         66.2MB
hyperledger/fabric-peer:2.5.15              3e25adc31a75        230MB         66.2MB
hyperledger/fabric-peer:latest              3e25adc31a75        230MB         66.2MB
```

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples$ ./install-fabric.sh docker

Pull Hyperledger Fabric docker images

FABRIC_IMAGES: peer orderer ccenv baseos
===> Pulling fabric Images
====>  ghcr.io/hyperledger/fabric-peer:2.5.15
2.5.15: Pulling from hyperledger/fabric-peer
Digest: sha256:3e25adc31a757ee19dd8a24afdc9ac5aa46a86b3cb56fc93068b0e594706d6aa
Status: Image is up to date for ghcr.io/hyperledger/fabric-peer:2.5.15
ghcr.io/hyperledger/fabric-peer:2.5.15
====>  ghcr.io/hyperledger/fabric-orderer:2.5.15
2.5.15: Pulling from hyperledger/fabric-orderer
Digest: sha256:9972c209be47f45e377a24c88f2e3d21fae4dc224751c68bdd33f2e7a603ffa8
Status: Image is up to date for ghcr.io/hyperledger/fabric-orderer:2.5.15
ghcr.io/hyperledger/fabric-orderer:2.5.15
====>  ghcr.io/hyperledger/fabric-ccenv:2.5.15
2.5.15: Pulling from hyperledger/fabric-ccenv
Digest: sha256:86dded7eda9268e2cbf3691cb952edf545dc5faea1a79bfc9b47ec47d3ecfa9e
Status: Image is up to date for ghcr.io/hyperledger/fabric-ccenv:2.5.15
ghcr.io/hyperledger/fabric-ccenv:2.5.15
```

**Explain:** This one script does three things simultaneously. docker pulls all Fabric Docker images — fabric-peer, fabric-orderer, fabric-ccenv, fabric-baseos, fabric-ca, and critically fabric-nodeenv (needed to build Node.js chaincode). binary downloads the CLI tools — peer, configtxgen, configtxlator — into ~/fabric-samples/bin. samples ensures the chaincode sample directories are present. Without fabric-nodeenv, chaincode deployment fails later.

**Step 5 — Verify Docker images are present**

**code :**

*docker images | grep fabric*

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples$ docker images | grep fabric
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15    654bd4554bf9        250MB          80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17        453a0a727e82        381MB           123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15     86dded7eda92          1GB           255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15   9972c209be47        178MB          49.5MB   U
ghcr.io/hyperledger/fabric-peer:2.5.15      3e25adc31a75        230MB          66.2MB   U
hyperledger/fabric-baseos:2.5               654bd4554bf9        250MB          80.4MB
hyperledger/fabric-baseos:2.5.15            654bd4554bf9        250MB          80.4MB
hyperledger/fabric-baseos:latest            654bd4554bf9        250MB          80.4MB
hyperledger/fabric-ca:1.5                   453a0a727e82        381MB           123MB
hyperledger/fabric-ca:1.5.17                453a0a727e82        381MB           123MB
hyperledger/fabric-ca:latest                453a0a727e82        381MB           123MB
hyperledger/fabric-ccenv:2.5                86dded7eda92          1GB           255MB
hyperledger/fabric-ccenv:2.5.15             86dded7eda92          1GB           255MB
hyperledger/fabric-ccenv:latest             86dded7eda92          1GB           255MB
hyperledger/fabric-nodeenv:2.5              17e2d447ca0d        570MB           146MB
hyperledger/fabric-orderer:2.5              9972c209be47        178MB          49.5MB   U
hyperledger/fabric-orderer:2.5.15           9972c209be47        178MB          49.5MB   U
hyperledger/fabric-orderer:latest           9972c209be47        178MB          49.5MB   U
hyperledger/fabric-peer:2.5                 3e25adc31a75        230MB          66.2MB   U
hyperledger/fabric-peer:2.5.15              3e25adc31a75        230MB          66.2MB   U
hyperledger/fabric-peer:latest              3e25adc31a75        230MB          66.2MB   U
```

**Explain:** Confirm hyperledger/fabric-nodeenv:2.5 is in the list. This image is the Node.js build environment that Fabric uses to compile and run JavaScript chaincode inside a container. If it is missing, deployCC will fail with a "No such image" error.

**Step 6 — Set PATH and config path**

**code :**

```
export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
```

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples$ export PATH=$PATH:$HOME/fabric-samples/bin
pranav@DESKTOP-OSHSLJI:~/fabric-samples$ export FABRIC_CFG_PATH=$HOME/fabric-samples/config
pranav@DESKTOP-OSHSLJI:~/fabric-samples$ peer version
peer:
 Version: v2.5.15
 Commit SHA: 83c7930
 Go version: go1.26.0
 OS/Arch: linux/amd64
 Chaincode:
  Base Docker Label: org.hyperledger.fabric
  Docker Namespace: hyperledger
```

**Explain:** The peer binary lives in ~/fabric-samples/bin, which is not on your shell's PATH by default. Without this export, the shell cannot find the command and returns "peer not found". FABRIC_CFG_PATH tells the peer CLI where to find core.yaml, the main configuration file. Without it, the peer errors with "cannot init crypto, path does not exist".

**code :**

*peer version*

You should see Version: v2.5.15.

**Step 7 — Start the network and create the channel**

**code :**

*cd ~/fabric-samples/test-network*

*./network.sh up createChannel*

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ cd ~/fabric-samples/test-network
./network.sh down
Using docker and docker-compose
Stopping network
[+] down 7/7
 ✔Container peer0.org1.example.com      Removed
 ✔Container orderer.example.com         Removed
 ✔Container peer0.org2.example.com      Removed
 ✔Network fabric_test                   Removed
 ✔Volume compose_orderer.example.com    Removed
 ✔Volume compose_peer0.org1.example.com Removed
 ✔Volume compose_peer0.org2.example.com Removed
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59
f864cfa1398b52d341cb:latest
Deleted: sha256:27056269ed9759828700d7bfc21149ac76c17d725ae6f9a12c07cc42a04f539f
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59
163835f604aa885d60dc:latest
Deleted: sha256:f7c74fe5fd5a6a73117e649e923a5de0a595771f793ee8e1b0047fc35d2d315c
```

```
pranav@DESKTOP-USHSLJ1:~/fabric-samples/test-network$ ./network.sh up createChannel
Using docker and docker-compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using dat
Bringing up network
LOCAL_VERSION=v2.5.15
DOCKER_IMAGE_VERSION=v2.5.15
/home/pranav/fabric-samples/test-network/../bin/cryptogen
Generating certificates using cryptogen tool
Creating Org1 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org1.yaml --output=organizations
org1.example.com
+ res=0
Creating Org2 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org2.yaml --output=organizations
org2.example.com
+ res=0
Creating Orderer Org Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-orderer.yaml --output=organizations
+ res=0
Generating CCP files for Org1 and Org2
[+] up 7/7
 ✔Network fabric_test                    Created
 ✔Volume compose_orderer.example.com     Created
 ✔Volume compose_peer0.org1.example.com  Created
 ✔Volume compose_peer0.org2.example.com  Created
 ✔Container peer0.org1.example.com       Created
 ✔Container orderer.example.com          Created
 ✔Container peer0.org2.example.com       Created
CONTAINER ID   IMAGE                                  COMMAND                  CREATED          STATUS
```

**Explain:** This single command does everything needed to have a running network with a channel. Internally it uses cryptogen to generate TLS certificates and identity material (MSP) for Org1, Org2, and the Orderer. It starts three Docker containers — orderer.example.com on port 7050, peer0.org1.example.com on port 7051, and peer0.org2.example.com on port 9051. It then runs configtxgen to create mychannel.block (the channel genesis block), submits it to the orderer, joins both peers to the channel, and sets anchor peers so the two organizations can discover each other via gossip. The default channel name is mychannel.

**Step 8 — Deploy the chaincode**

**code :**

```
./network.sh deployCC \
  -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript \
  -ccl javascript
```

```
pranav@DESKTOP-USHSLJ1:~/fabric-samples/test-network$ ./network.sh deployCC \
-ccn basic \
-ccp ../asset-transfer-basic/chaincode-javascript \
-ccl javascript
Using docker and docker-compose
Deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/chaincode-javascript --lang
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1
```

```
2026-03-17 16:28:58.033 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [b424caca912525c6970bb1da73f421e25a00030499eab767783fa7e
atus (VALID) at localhost:7051
2026-03-17 16:28:58.037 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [b424caca912525c6970bb1da73f421e25a00030499eab767783fa7e
atus (VALID) at localhost:9051
Chaincode definition committed on channel 'mychannel'
Using organization 1
Querying chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to Query committed status on peer0.org1, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to Query committed status on peer0.org2, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
Chaincode initialization is not required
```

**Explain:** -ccn basic is the name your chaincode will be called by in all future commands. -ccp is the path to the chaincode source code. -ccl javascript tells Fabric to use the Node.js runtime. Internally this runs the full Fabric chaincode lifecycle — package the source into a .tar.gz, install it on both peers, have Org1 approve the definition, have Org2 approve the definition (both approvals are required since majority endorsement is the default policy), and finally commit the definition to the channel. Once committed, two new Docker containers appear — one chaincode container per peer (dev-peer0.org1... and dev-peer0.org2...).

### Step 9 — Set peer identity environment variables

**code :**

*export FABRIC_CFG_PATH=$HOME/fabric-samples/config*
*export PATH=$PATH:$HOME/fabric-samples/bin*
*export CORE_PEER_TLS_ENABLED=true*
*export CORE_PEER_LOCALMSPID="Org1MSP"*
*export*
*CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt*
*export*
*CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp*
*export CORE_PEER_ADDRESS=localhost:7051*

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config

export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.o
t
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.
export CORE_PEER_ADDRESS=localhost:7051
```

**Explain:** The peer CLI has no built-in session — it does not know who you are. Every command needs to know which organization's identity to use, which peer to connect to, and how to verify TLS. These five variables provide exactly that. CORE_PEER_LOCALMSPID declares "I am Org1". CORE_PEER_MSPCONFIGPATH points to the Admin user's private signing key and certificate. CORE_PEER_TLS_ROOTCERT_FILE provides the CA certificate to verify the

peer's TLS identity. CORE_PEER_ADDRESS is which peer to send requests to. These must be re-exported in every new terminal session.

**Step 10 — Initialize the ledger with sample data**

**code :**

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/
peer0.org1.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/
peer0.org2.example.com/tls/ca.crt \
  --waitForEvent \
  -c '{"function":"InitLedger","Args":[]}'
```

```
pranav@DESKTOP-USHSLJ1:~/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscac
.pem \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.co
--peerAddresses localhost:9051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.co
--waitForEvent \
-c '{"function":"InitLedger","Args":[]}'
2026-03-17 16:37:35.757 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [f982da882feb2411792ddb2c62b47591512c0269f789efe8ab9ce462
atus (VALID) at localhost:9051
2026-03-17 16:37:35.757 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [f982da882feb2411792ddb2c62b47591512c0269f789efe8ab9ce462
atus (VALID) at localhost:7051
2026-03-17 16:37:35.758 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
```

**Explain:** invoke is used for any operation that writes to the ledger. -o localhost:7050 is the orderer address. --tls and --cafile enable TLS and provide the CA cert to verify the orderer. -C mychannel and -n basic identify which channel and chaincode. The two --peerAddresses and --tlsRootCertFiles pairs are the critical part — the default endorsement policy requires a majority of organizations to endorse. With two orgs, both must sign. Sending to only one peer results in ENDORSEMENT_POLICY_FAILURE. --waitForEvent blocks until both peers confirm the

transaction is committed (status VALID) rather than just submitted. InitLedger seeds the ledger with six sample assets.

**Step 11 — Query all assets**

**code :**

```
peer chaincode query \
  -C mychannel \
  -n basic \
  -c '{"Args":["GetAllAssets"]}'
```

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"},{"AppraisedValue":400,"Color":"red","ID":"a
Brad","Size":5,"docType":"asset"},{"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"},{"Appra
Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"},{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adrian
cType":"asset"},{"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
```

**Explain:** query is for read-only operations. No transaction is created, no block is written, no endorsement from multiple peers is needed. It is fast and cheap. This returns all six assets seeded by InitLedger:


```
[ {"ID":"asset1","Color":"blue","Size":5,"Owner":"Tomoko","AppraisedValue":300},
  {"ID":"asset2","Color":"red","Size":5,"Owner":"Brad","AppraisedValue":400},
  {"ID":"asset3","Color":"green","Size":10,"Owner":"Jin Soo","AppraisedValue":500},
  {"ID":"asset4","Color":"yellow","Size":10,"Owner":"Max","AppraisedValue":600},
  {"ID":"asset5","Color":"black","Size":15,"Owner":"Adriana","AppraisedValue":700},
  {"ID":"asset6","Color":"white","Size":15,"Owner":"Michel","AppraisedValue":800} ]
```


**Step 12 — Read a specific asset**

**code :**

```
peer chaincode query \
  -C mychannel \
  -n basic \
  -c '{"Args":["ReadAsset","asset5"]}'
```

```
{"AppraisedValue":1000,"Color":"blue","ID":"asset10","Owner":"Pranav","Size":10}
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}
```

Result:


{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15}


**Explain:** ReadAsset takes an asset ID and returns that single asset's data from the world state. The world state is a key-value database that stores the current (latest) value for each asset. The full transaction history is stored in the blockchain, but queries read from this faster world state.

**Step 13 — Create a new asset**

**code :**

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/
peer0.org1.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/
peer0.org2.example.com/tls/ca.crt \
  --waitForEvent \
  -c '{"function":"CreateAsset","Args":["asset10","blue","10","Pranav","1000"]}'
```

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tls
.pem \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example
--peerAddresses localhost:9051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example
--waitForEvent \
-c '{"function":"CreateAsset","Args":["asset10","blue","10","Pranav","1000"]}'
2026-03-17 16:38:30.370 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [d88abbf8b7aaff8416fb4a8914a4d01c8064d053ebc5145787034
atus (VALID) at localhost:7051
2026-03-17 16:38:30.371 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [d88abbf8b7aaff8416fb4a8914a4d01c8064d053ebc5145787034
atus (VALID) at localhost:9051
2026-03-17 16:38:30.371 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
\"Color\":\"blue\",\"Size\":10,\"Owner\":\"Pranav\",\"AppraisedValue\":1000}"
```

**Explain:** CreateAsset takes five arguments in order — [ID, Color, Size, Owner, AppraisedValue]. This is a write operation so both peers must endorse. The orderer packages the endorsed transaction into a block and delivers it to both peers. Both confirm VALID and the asset is permanently recorded on the ledger.

**code :**

peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset10"]}'

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args":["ReadAsset","asset10"]}'
{"AppraisedValue":1000,"Color":"blue","ID":"asset10","Owner":"Pranav","Size":10}
```

# {"AppraisedValue":1000,"Color":"blue","ID":"asset10","Owner":"Pranav","Size":10}

The transaction flow showing what happens inside Fabric every time you run invoke or query Shows what happens inside Fabric every time you run peer chaincode invoke — the proposal, endorsement, ordering, and commit flow — and why both peers are always required for any write operation.

**Step 14 — Transfer asset ownership**

**code :**

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/
peer0.org1.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/
peer0.org2.example.com/tls/ca.crt \
  --waitForEvent \
  -c '{"function":"TransferAsset","Args":["asset10","Rahul"]}'
```

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.c
.pem \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
--waitForEvent \
-c '{"function":"TransferAsset","Args":["asset10","Rahul"]}'
2026-03-17 16:41:28.692 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [e834ab1020eac67d6bbf5759291d3c3bd92083d1a340d1f5bd35104c2e387019] committed
atus (VALID) at localhost:9051
2026-03-17 16:41:28.692 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [e834ab1020eac67d6bbf5759291d3c3bd92083d1a340d1f5bd35104c2e387019] committed
atus (VALID) at localhost:7051
2026-03-17 16:41:28.692 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"Pranav"
```

**Explain:** TransferAsset takes [assetID, newOwner]. It reads the current asset from the world state, updates the Owner field, and writes it back. The function returns the previous owner ("Pranav") as confirmation. Both peers endorse, the orderer commits the block, and the ledger is updated permanently.

**Verify the ownership change:**

**code :**

*peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset10"]}'*

# {"AppraisedValue":1000,"Color":"blue","ID":"asset10","Owner":"Rahul","Size":10}

```
2026-03-17 16:41:28.692 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"Pranav"
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset10"]}'
{"AppraisedValue":1000,"Color":"blue","ID":"asset10","Owner":"Rahul","Size":10}
```

Owner is now **Rahul**

**Step 15 — To stop/bring down the Hyperledger Fabric network, run:**

*./network.sh down*

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ ./network.sh down
Using docker and docker-compose
Stopping network
[+] down 7/7
 ✔ Container orderer.example.com          Removed
 ✔ Container peer0.org2.example.com       Removed
 ✔ Container peer0.org1.example.com       Removed
 ✔ Network fabric_test                    Removed
 ✔ Volume compose_peer0.org1.example.com  Removed
 ✔ Volume compose_peer0.org2.example.com  Removed
 ✔ Volume compose_orderer.example.com     Removed
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035
f864cfa1398b52d341cb:latest
Deleted: sha256:aa29d3b9fb04e6316131fbf334a23f28d8a1646fa9d40090916c02d54bc5b76d
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035
163835f604aa885d60dc:latest
Deleted: sha256:08d49c295856f950367880744bcacfaf56a7134bf208cacfab8a29ef9221703c
```

**Conclusion**

The Hyperledger Fabric network was successfully implemented, demonstrating asset creation, querying, and transfer using chaincode. The process highlights key concepts such as consensus, immutability, and secure transaction flow. Overall, it provides a clear understanding of permissioned blockchain systems and their real-world applications in asset management.